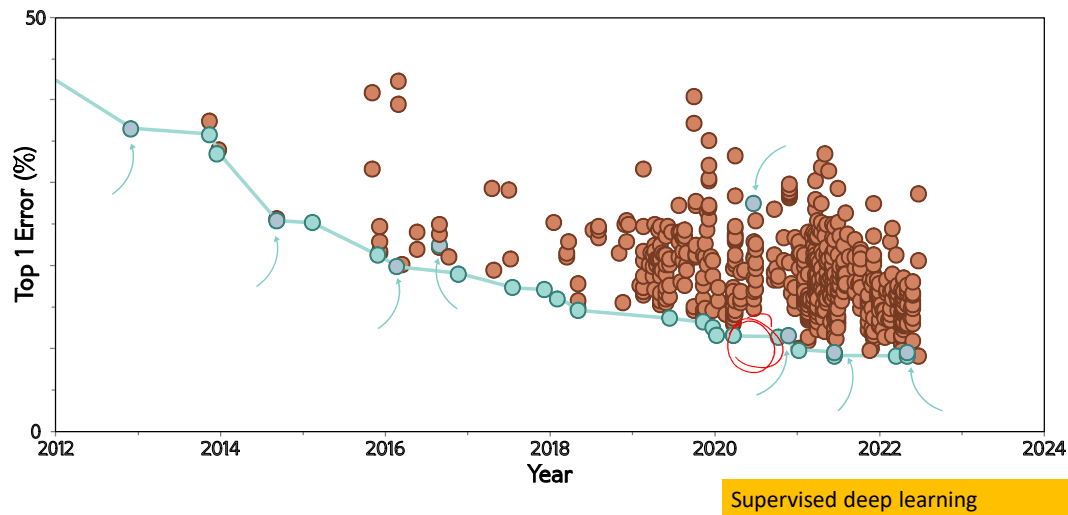


Image Classification Recent History

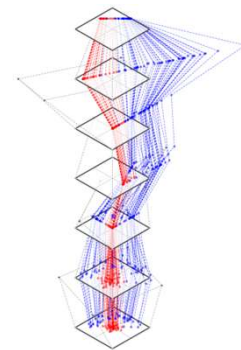


1

Artificial Neural Nets

○ ANNs are a particular (simple) way of fixing the hypothesis, and a consistent way of optimizing

- They are universal approximators
- Can be trained using a simple algorithm (stochastic gradient descent)
- Run efficiently on parallel hardware.
- Inductive biases
- Build useful abstraction



2

Image classification

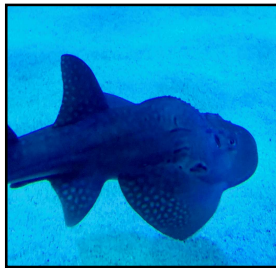
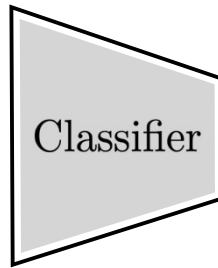


image x



Classifier



Guitarfish

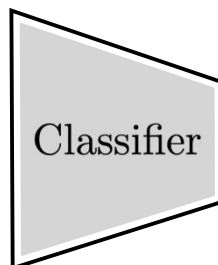
label y

3

Image classification



image x



Classifier



Night heron

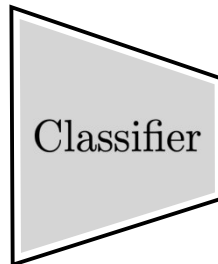
label y

4

Image classification



image x



Bumble bee

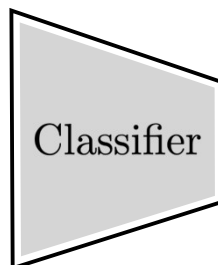
label y

5

Image classification



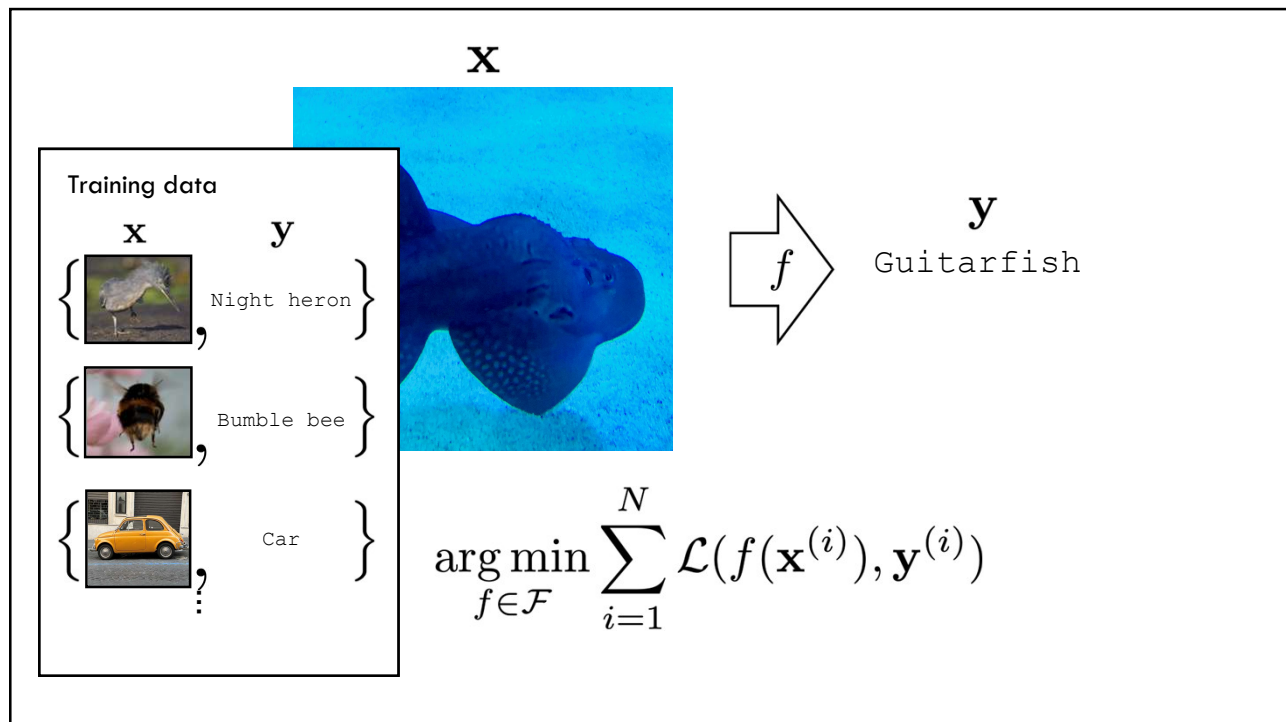
image x



Car

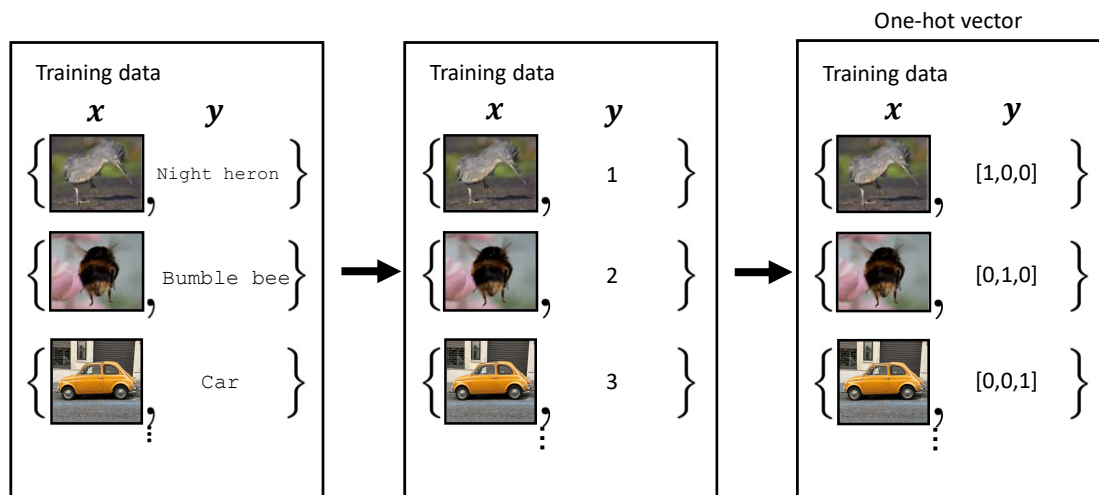
label y

6



7

How to represent class labels?



8

What should the loss function be?

- Training dataset of I pairs of input/output examples:

$$\{\mathbf{x}_i, \mathbf{y}_i\}_{i=1}^I$$

- Loss function** or **cost function** measures how bad model is:

$$L \left[\underbrace{\phi}_{\text{model}}, \underbrace{f[\mathbf{x}, \phi], \{\mathbf{x}_i, \mathbf{y}_i\}_{i=1}^I}_{\text{train data}} \right]$$

9

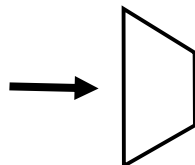
What should the loss be?

0-1 loss (number of misclassifications)

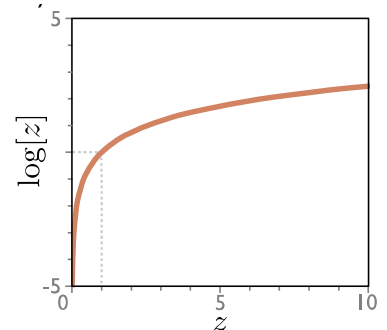
$$\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y}) = \mathbb{1}(\hat{\mathbf{y}} \neq \mathbf{y}) \quad \leftarrow \text{discrete, NP-hard to optimize!}$$

1 instance but
K classes

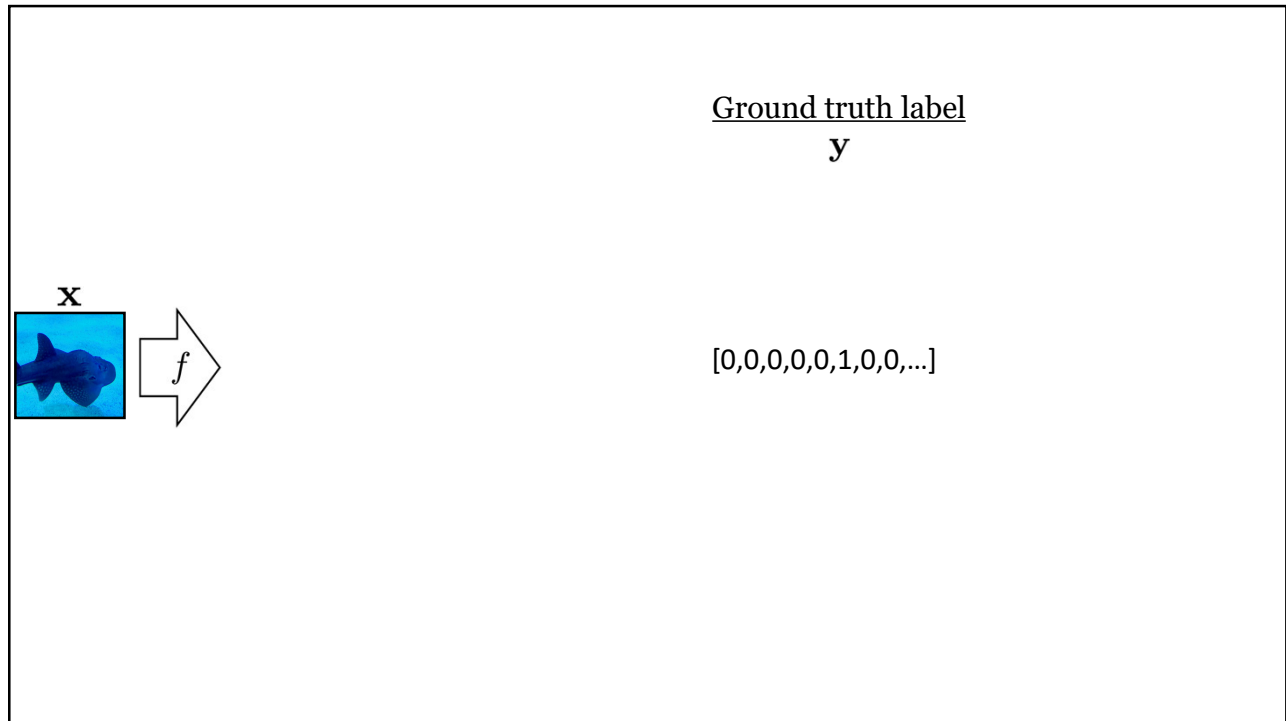
$$\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y}) = H(\mathbf{y}, \hat{\mathbf{y}}) = - \sum_{k=1}^K y_k \log \hat{y}_k$$



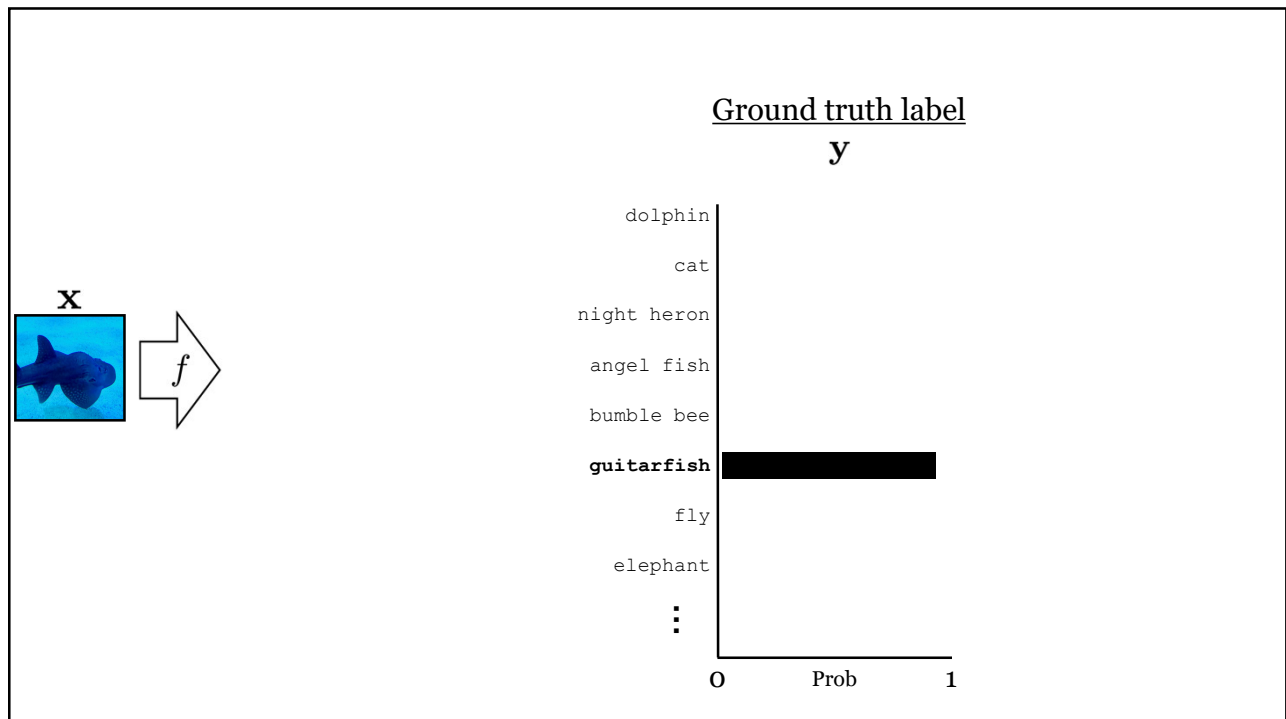
Guitarfish



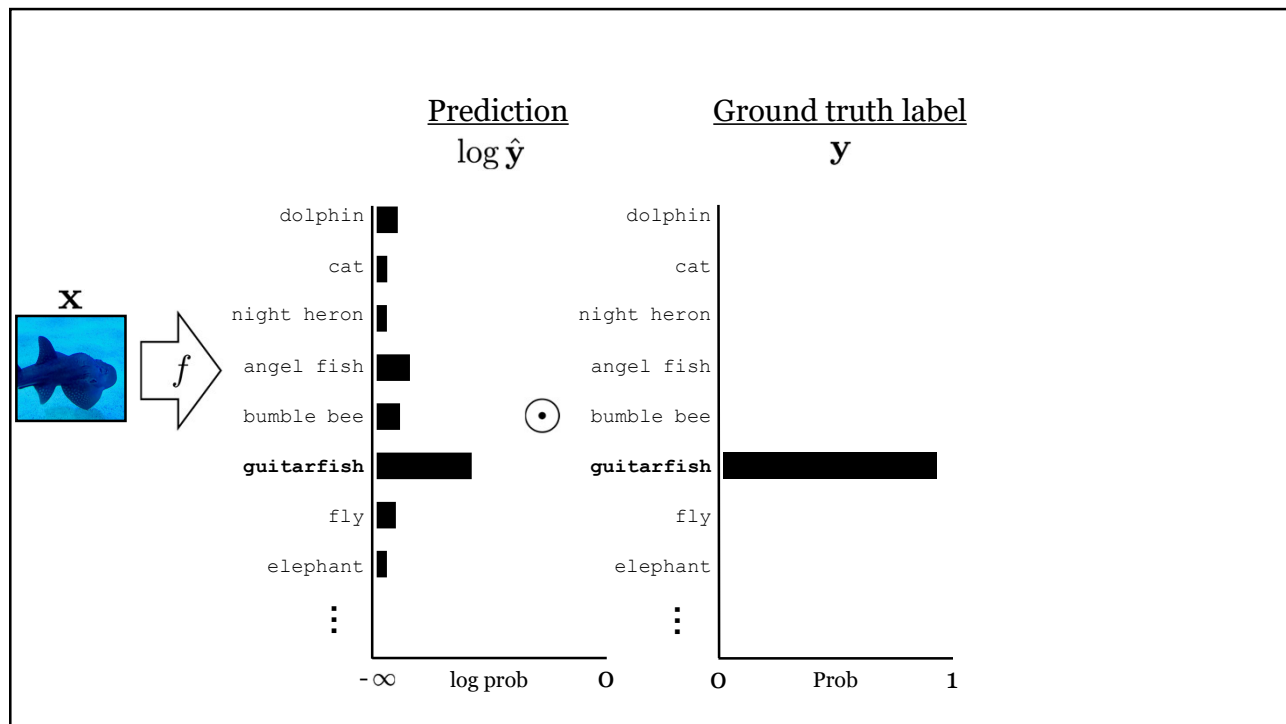
10



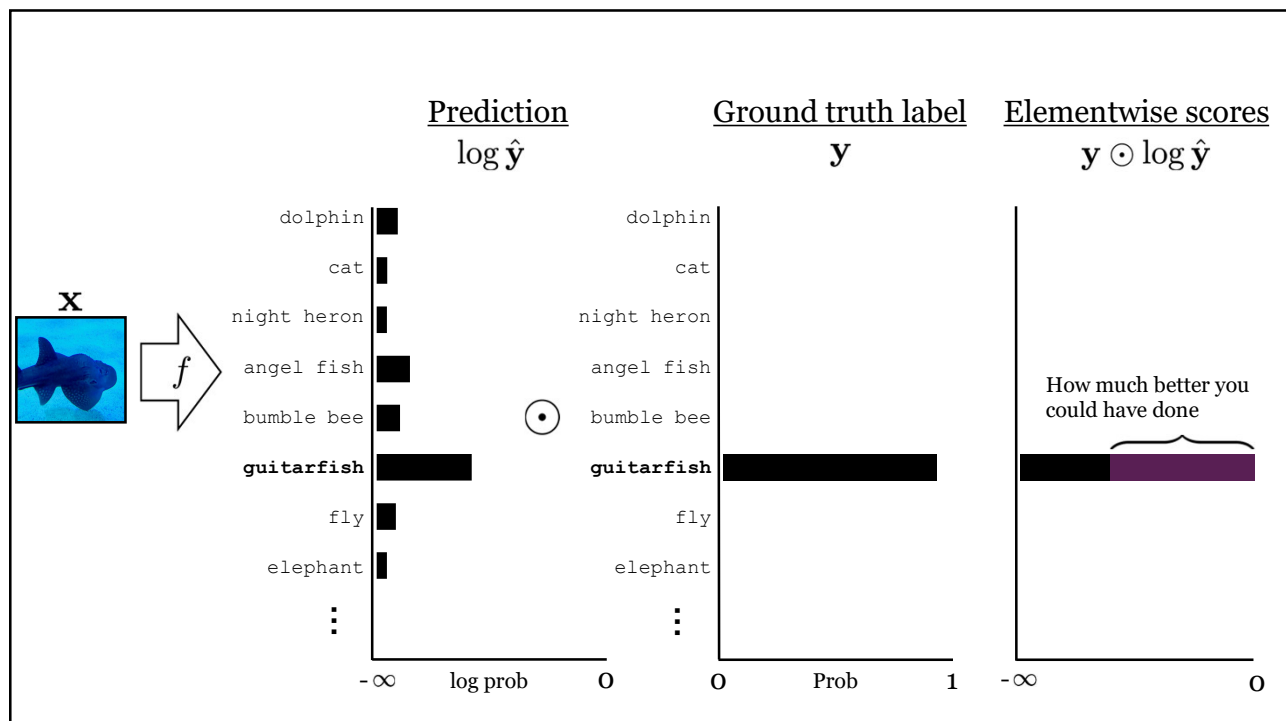
11



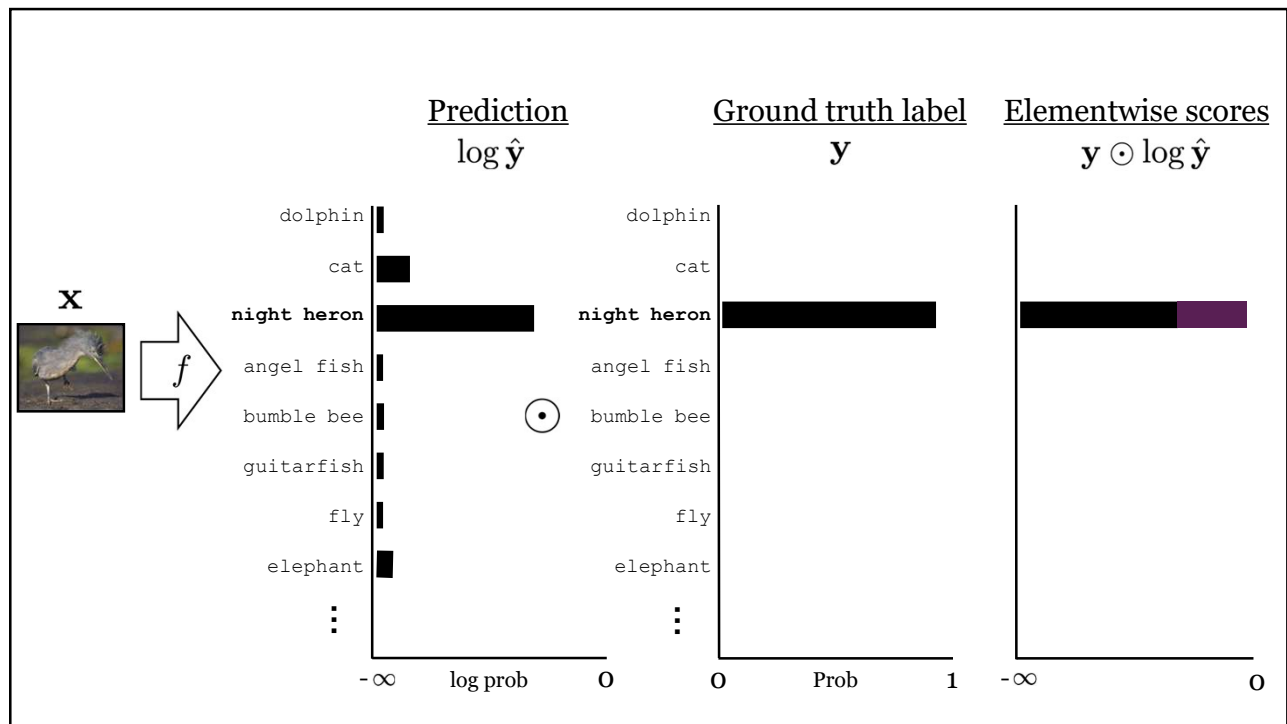
12



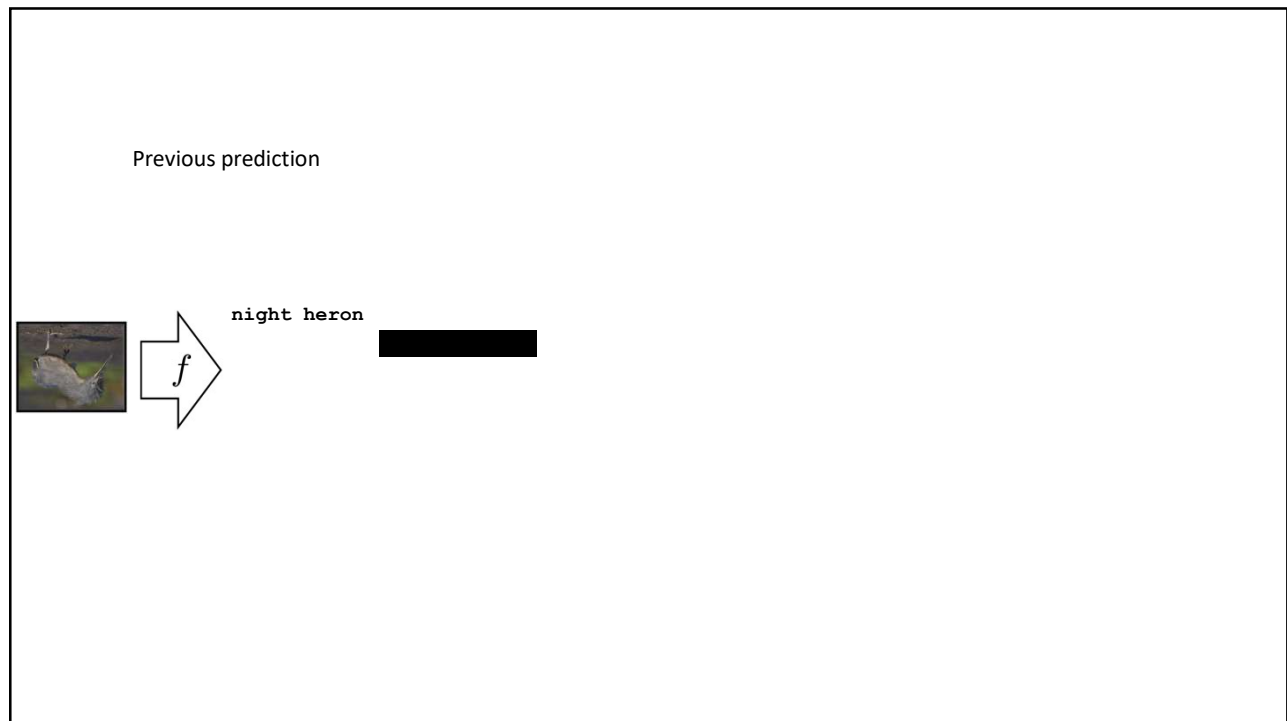
13



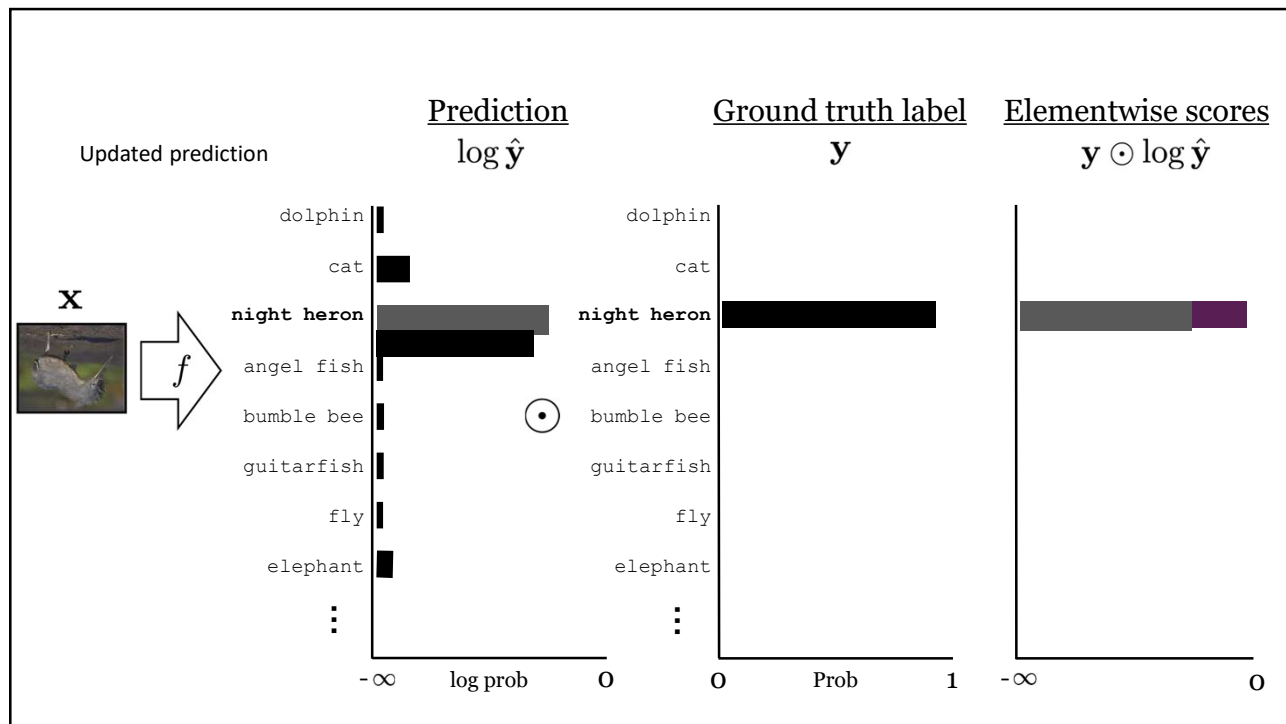
14



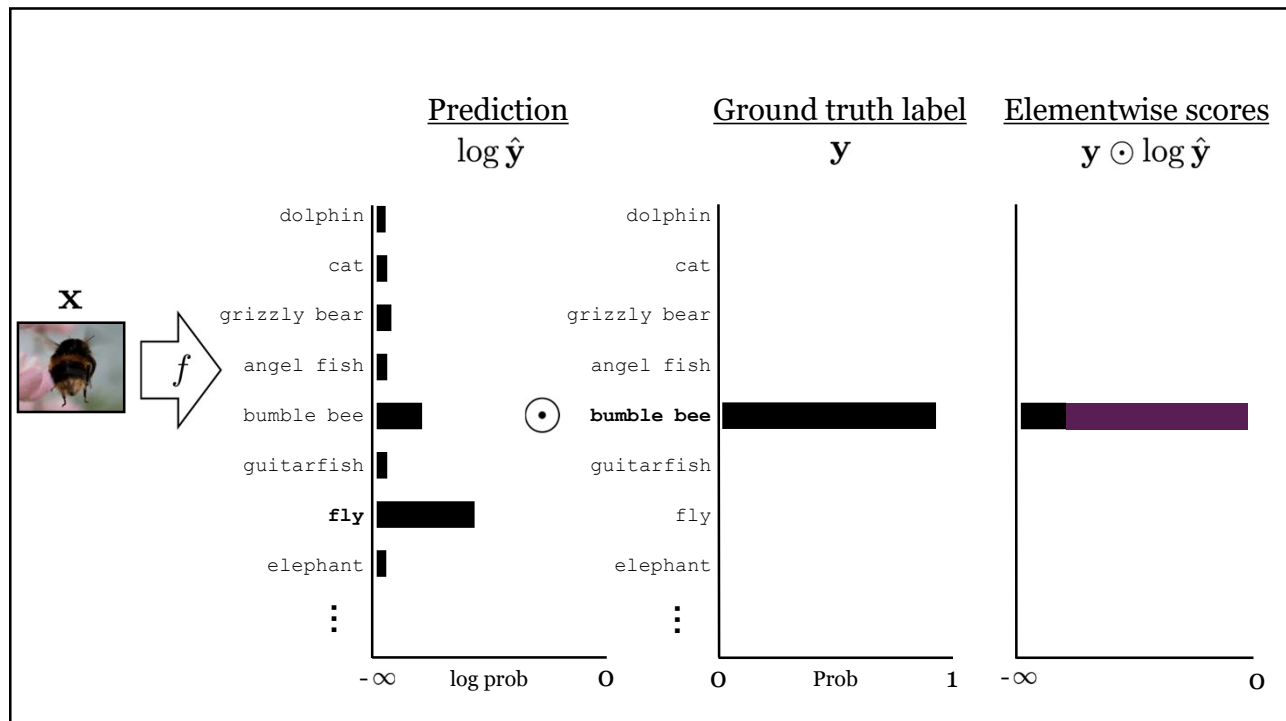
15



16



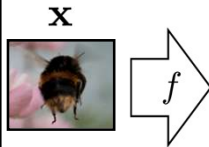
17



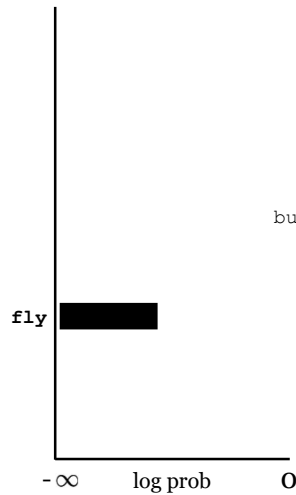
18

Binary classification

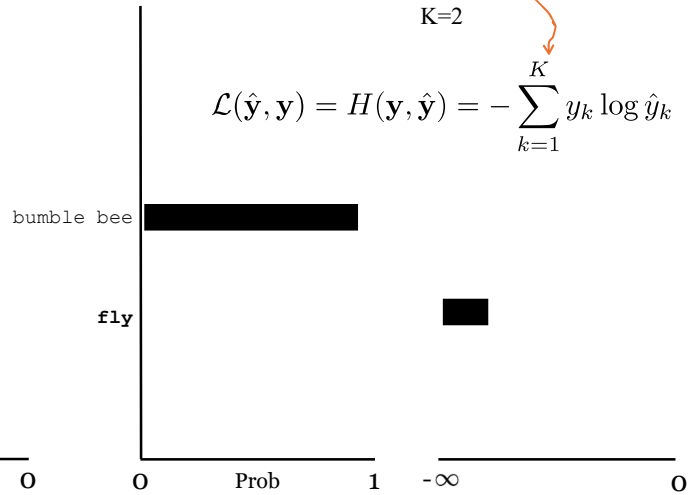
Imagine binary classification:
Bumble bee or fly?



Model predicts fly



Ground Truth is bumble bee



What is the loss?

$K=2$

$$\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y}) = H(\mathbf{y}, \hat{\mathbf{y}}) = - \sum_{k=1}^K y_k \log \hat{y}_k$$

19

sigmoid

Problem:

- Output of neural network can be anything

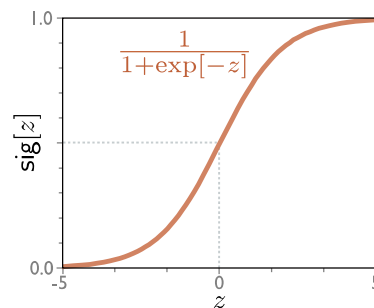
Solution:

- Pass through logistic sigmoid function that maps "anything to [0,1]:

$$\text{sig}[z] = \frac{1}{1 + \exp[-z]}$$

I instance examples
("mini batch")

$$L[\phi] = \sum_{i=1}^I -(1 - y_i) \log [1 - \text{sig}[f[\mathbf{x}_i|\phi]]] - y_i \log [\text{sig}[f[\mathbf{x}_i|\phi]]]$$



Here the model outputs only 1 score unlike the previous pictures seen earlier since we are doing binary classification

20

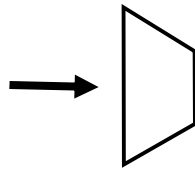
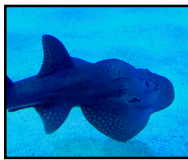
What should the loss be?

0-1 loss (number of misclassifications)

$$\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y}) = \mathbb{1}(\hat{\mathbf{y}} \neq \mathbf{y}) \quad \leftarrow \text{discrete, NP-hard to optimize!}$$

This is called **minimizing cross entropy**

$$\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y}) = H(\mathbf{y}, \hat{\mathbf{y}}) = - \sum_{k=1}^K y_k \log \hat{y}_k \quad \leftarrow H(p, q) = - \left(\sum_{i=1}^n p_i \log q_i \right)$$



Guitarfish

22

Information, Entropy and Cross Entropy

$1 \rightarrow \mathcal{I}(p) = -\log p$
 $\log_2 \frac{1}{2} = \log_2 2^{-1} = -(-1) = 1$
 1-bit inform
 $\frac{0}{4}$

$X \quad \begin{matrix} X_1 \rightarrow P_1 \\ X_2 \rightarrow P_2 \end{matrix}$
 Entropy $H(X) = - \sum P_i \log p_i$
 $P = (0.9, 0.1) \leftarrow$
 Entropy $-(0.9 \log 0.9 + 0.1 \log 0.1) \approx 0.469$

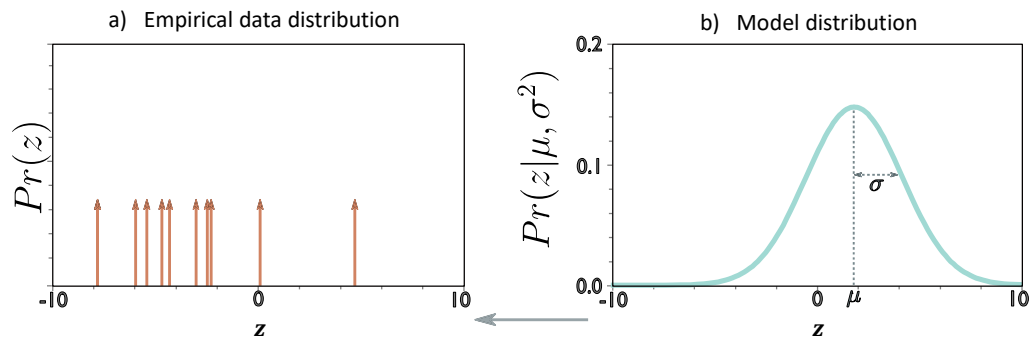
Suppose $q = (0.6, 0.4)$
 Cross entropy $H(\underline{P}, q) = -(P_1 \log q_1 + P_2 \log q_2)$
 $\text{Example } = -(0.9 \log 0.6 + 0.1 \log 0.4) \approx 0.7 \rightarrow 0.7 - 0.4 = 0.3$

Cross Entropy $\geq$ The Entropy
 $0.7 - 0.4 = 0.3$

23

KL Divergence

In the previous handwritten slide, we have interchanged p and q , i.e. p (true) and q (model)



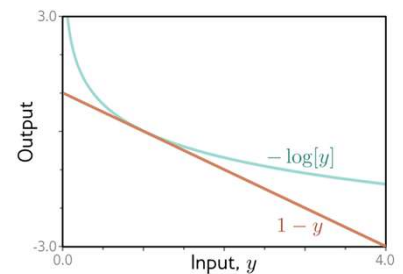
$$\text{KL}[q||p] = \int_{-\infty}^{\infty} q(z) \log[q(z)] dz - \int_{-\infty}^{\infty} q(z) \log[p(z)] dz$$

Kullback-Leibler Divergence -- a measure between probability distributions

24

KL Divergence is always positive

$$\begin{aligned} D_{KL}[p(x)||q(x)] &= \int p(x) \log \left[\frac{p(x)}{q(x)} \right] dx \\ &= - \int p(x) \log \left[\frac{q(x)}{p(x)} \right] dx \\ &\geq \int p(x) \left(1 - \frac{q(x)}{p(x)} \right) dx \\ &= \int p(x) - q(x) dx \\ &= 1 - 1 = 0. \end{aligned}$$



Cross entropy is equal to the true entropy plus the divergence, so minimizing the KL divergence is the same as minimizing the cross entropy

25

Minimizing cross entropy

$$\begin{aligned}\hat{\theta} &= \operatorname{argmin}_{\theta} \left[\int_{-\infty}^{\infty} q(y) \log[q(y)] dy - \int_{-\infty}^{\infty} q(y) \log[Pr(y|\theta)] dy \right] \\ &= \operatorname{argmin}_{\theta} \left[- \int_{-\infty}^{\infty} q(y) \log[Pr(y|\theta)] dy \right],\end{aligned}$$

$$\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y}) = H(\mathbf{y}, \hat{\mathbf{y}}) = - \sum_{k=1}^K y_k \log \hat{y}_k$$

When we minimize loss we are minimizing the KL divergence and hence minimizing the cross entropy

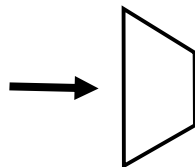
26

Minimizing log likelihood

Why is this also called minimizing log likelihood also?

This is called ~~cross-entropy~~ minimizing log likelihood

$$\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y}) = H(\mathbf{y}, \hat{\mathbf{y}}) = - \sum_{k=1}^K y_k \log \hat{y}_k \quad \leftarrow$$



→ Guitarfish

27

Loss functions

- ~~Model predicts output y given input x~~

28

loss functions

- ~~Model predicts output y given input x~~
- Model predicts a conditional probability distribution:

$$Pr(\mathbf{y}|\mathbf{x})$$

over outputs $\mathbf{y}$ given inputs $\mathbf{x}$.

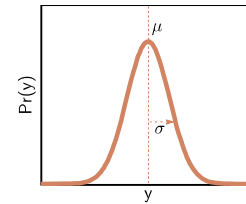
- Loss function aims to make the outputs have high probability

29

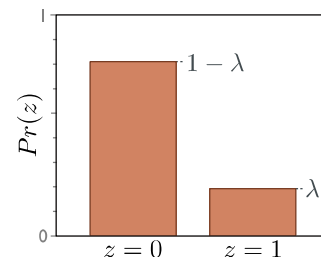
How a model predicts a probability distribution?

1. Pick a known distribution (e.g., normal distribution or a Bernoulli distribution) to model output y with parameters
2. Use model to predict parameters of probability distribution

$$\theta = \{\mu, \sigma^2\}$$



$$\theta = \{\lambda\}$$



30

Maximum likelihood criterion

$$\begin{aligned}\hat{\phi} &= \operatorname{argmax}_{\phi} \left[\prod_{i=1}^I \operatorname{Pr}(\mathbf{y}_i | \mathbf{x}_i) \right] \\ &= \operatorname{argmax}_{\phi} \left[\prod_{i=1}^I \operatorname{Pr}(\mathbf{y}_i | \theta_i) \right] \\ &= \operatorname{argmax}_{\phi} \left[\prod_{i=1}^I \operatorname{Pr}(\mathbf{y}_i | \mathbf{f}[\mathbf{x}_i, \phi]) \right]\end{aligned}$$

When we consider this probability as a function of the parameters ϕ , we call it a **likelihood**.

31

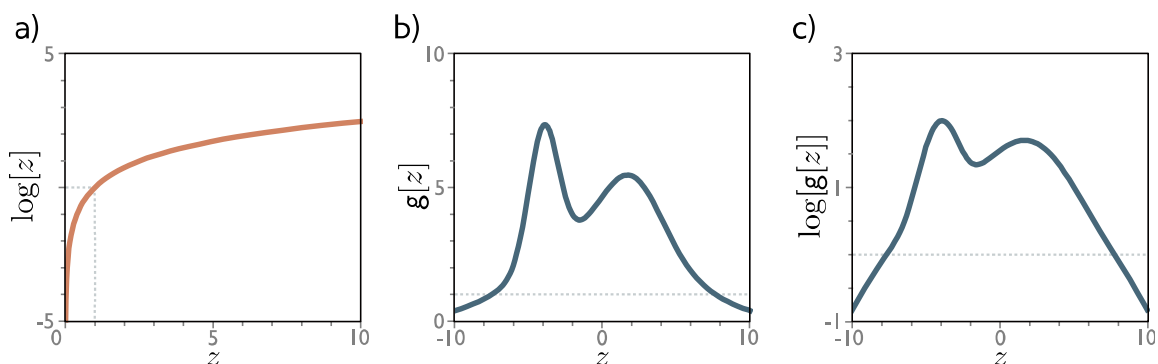
Problem:

$$\hat{\phi} = \operatorname{argmax}_{\phi} \left[\prod_{i=1}^I \operatorname{Pr}(\mathbf{y}_i | \mathbf{f}[\mathbf{x}_i, \phi]) \right]$$

- The terms in this product might all be small
- The product might get so small that we can't easily represent it

32

The log function is monotonic



Maximum of the logarithm of a function is in the same place as maximum of function

33

Maximum log likelihood

$$\begin{aligned}
 \hat{\phi} &= \operatorname{argmax}_{\phi} \left[\prod_{i=1}^I \operatorname{Pr}(\mathbf{y}_i | \mathbf{f}[\mathbf{x}_i, \phi]) \right] \\
 &= \operatorname{argmax}_{\phi} \left[\log \left[\prod_{i=1}^I \operatorname{Pr}(\mathbf{y}_i | \mathbf{f}[\mathbf{x}_i, \phi]) \right] \right] \\
 &= \operatorname{argmax}_{\phi} \left[\sum_{i=1}^I \log \left[\operatorname{Pr}(\mathbf{y}_i | \mathbf{f}[\mathbf{x}_i, \phi]) \right] \right]
 \end{aligned}$$

Now it's a sum of terms, so doesn't matter so much if the terms are small

34

Minimizing negative log likelihood

- By convention, we minimize things (i.e., a loss)

$$\begin{aligned}
 \hat{\phi} &= \operatorname{argmax}_{\phi} \left[\sum_{i=1}^I \log \left[\operatorname{Pr}(\mathbf{y}_i | \mathbf{f}[\mathbf{x}_i, \phi]) \right] \right] \\
 &= \operatorname{argmin}_{\phi} \left[- \sum_{i=1}^I \log \left[\operatorname{Pr}(\mathbf{y}_i | \mathbf{f}[\mathbf{x}_i, \phi]) \right] \right] \\
 &= \operatorname{argmin}_{\phi} \left[L[\phi] \right]
 \end{aligned}$$

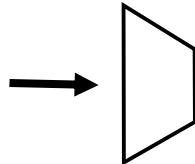
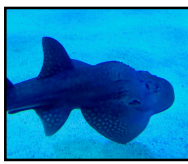
35

Why is this MLE

$$\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y}) = H(\mathbf{y}, \hat{\mathbf{y}}) = - \sum_{k=1}^K y_k \log \hat{y}_k$$

$$\hat{\phi} = \operatorname{argmax}_{\phi} \left[\sum_{i=1}^I \log \left[\operatorname{Pr}(\mathbf{y}_i | \mathbf{f}[\mathbf{x}_i, \phi]) \right] \right]$$

$$= \operatorname{argmin}_{\phi} \left[- \sum_{i=1}^I \log \left[\operatorname{Pr}(\mathbf{y}_i | \mathbf{f}[\mathbf{x}_i, \phi]) \right] \right]$$



Guitarfish

37

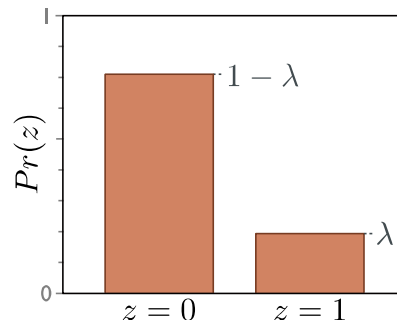
Consider Binary classification again

1. Choose a suitable probability distribution $\operatorname{Pr}(\mathbf{y}|\boldsymbol{\theta})$ that is defined over the domain of the predictions $\mathbf{y}$ and has distribution parameters $\boldsymbol{\theta}$.

- Domain: $y \in \{0, 1\}$
- Bernoulli distribution
- One parameter $\lambda \in [0, 1]$

$$\operatorname{Pr}(y|\lambda) = \begin{cases} 1 - \lambda & y = 0 \\ \lambda & y = 1 \end{cases}$$

$$\operatorname{Pr}(y|\lambda) = (1 - \lambda)^{1-y} \cdot \lambda^y$$



38

Binary classification

2. Set the machine learning model $\mathbf{f}[\mathbf{x}, \phi]$ to predict one or more of these parameters so $\boldsymbol{\theta} = \mathbf{f}[\mathbf{x}, \phi]$ and $Pr(\mathbf{y}|\boldsymbol{\theta}) = Pr(\mathbf{y}|\mathbf{f}[\mathbf{x}, \phi])$.

Problem:

- Output of neural network can be anything
- Parameter $\lambda \in [0,1]$

Solution:

- Pass through function that maps “anything to $[0,1]$ ”

39

Binary classification

2. Set the machine learning model $\mathbf{f}[\mathbf{x}, \phi]$ to predict one or more of these parameters so $\boldsymbol{\theta} = \mathbf{f}[\mathbf{x}, \phi]$ and $Pr(\mathbf{y}|\boldsymbol{\theta}) = Pr(\mathbf{y}|\mathbf{f}[\mathbf{x}, \phi])$.

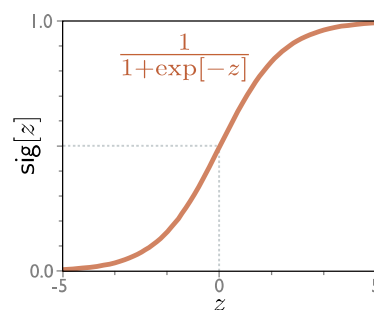
Problem:

- Output of neural network can be anything
- Parameter $\lambda \in [0,1]$

Solution:

- Pass through logistic sigmoid function that maps “anything to $[0,1]$ ”:

$$\text{sig}[z] = \frac{1}{1 + \exp[-z]}$$



40

Binary classification

2. Set the machine learning model $\mathbf{f}[\mathbf{x}, \phi]$ to predict one or more of these parameters so $\boldsymbol{\theta} = \mathbf{f}[\mathbf{x}, \phi]$ and $Pr(\mathbf{y}|\boldsymbol{\theta}) = Pr(\mathbf{y}|\mathbf{f}[\mathbf{x}, \phi])$.

$$Pr(y|\lambda) = (1 - \lambda)^{1-y} \cdot \lambda^y$$

$$Pr(y|\mathbf{x}) = (1 - \text{sig}[\mathbf{f}[\mathbf{x}|\phi]])^{1-y} \cdot \text{sig}[\mathbf{f}[\mathbf{x}|\phi]]^y$$

41

Binary classification

$$\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y}) = H(\mathbf{y}, \hat{\mathbf{y}}) = - \sum_{k=1}^K y_k \log \hat{y}_k$$

3. To train the model, find the network parameters $\hat{\phi}$ that minimize the negative log-likelihood loss function over the training dataset pairs $\{\mathbf{x}_i, \mathbf{y}_i\}$:

$$\hat{\phi} = \underset{\phi}{\operatorname{argmin}} [L[\phi]] = \underset{\phi}{\operatorname{argmin}} \left[- \sum_{i=1}^I \log [Pr(\mathbf{y}_i|\mathbf{f}[\mathbf{x}_i, \phi])] \right]. \quad (5.7)$$

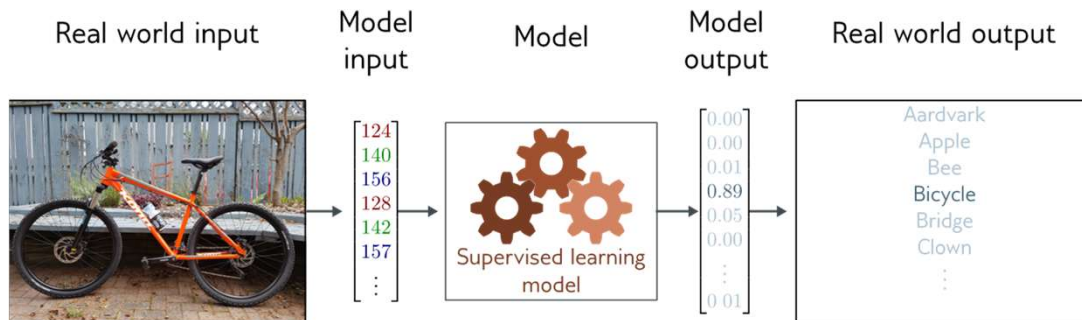
$$Pr(y|\mathbf{x}) = (1 - \text{sig}[\mathbf{f}[\mathbf{x}|\phi]])^{1-y} \cdot \text{sig}[\mathbf{f}[\mathbf{x}|\phi]]^y$$

$$L[\phi] = \sum_{i=1}^I -(1 - y_i) \log [1 - \text{sig}[\mathbf{f}[\mathbf{x}_i|\phi]]] - y_i \log [\text{sig}[\mathbf{f}[\mathbf{x}_i|\phi]]]$$

Binary cross-entropy loss

43

multiclass classification



Goal: predict which of K classes $y \in \{1, 2, \dots, K\}$ the input x belongs to

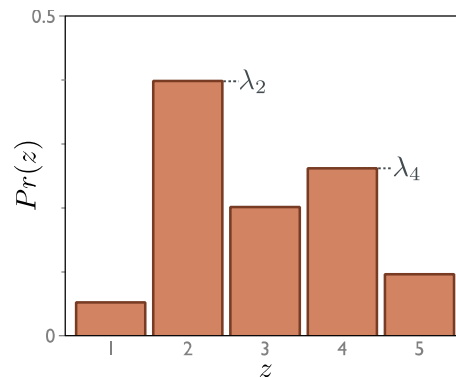
45

multiclass classification

1. Choose a suitable probability distribution $Pr(\mathbf{y}|\boldsymbol{\theta})$ that is defined over the domain of the predictions $\mathbf{y}$ and has distribution parameters $\boldsymbol{\theta}$.

- Domain: $y \in \{1, 2, \dots, K\}$
- Categorical distribution
- K parameters $\lambda_k \in [0, 1]$
- Sum of all parameters = 1

$$Pr(y = k) = \lambda_k$$



46

multiclass classification

2. Set the machine learning model $\mathbf{f}[\mathbf{x}, \phi]$ to predict one or more of these parameters so $\theta = \mathbf{f}[\mathbf{x}, \phi]$ and $Pr(\mathbf{y}|\theta) = Pr(\mathbf{y}|\mathbf{f}[\mathbf{x}, \phi])$.

Problem:

- Output of neural network can be anything
- Parameters $\lambda_k \in [0,1]$, sum to one

Solution:

- Pass through function that maps "anything" to $[0,1]$, sum to one

$$\text{softmax}_k[\mathbf{z}] = \frac{\exp[z_k]}{\sum_{k'=1}^K \exp[z_{k'}]}$$

$$Pr(y = k|\mathbf{x}) = \text{softmax}_k[\mathbf{f}[\mathbf{x}, \phi]]$$

47

multiclass classification

$$\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y}) = H(\mathbf{y}, \hat{\mathbf{y}}) = - \sum_{k=1}^K y_k \log \hat{y}_k$$

3. To train the model, find the network parameters $\hat{\phi}$ that minimize the negative log-likelihood loss function over the training dataset pairs $\{\mathbf{x}_i, \mathbf{y}_i\}$:

$$\hat{\phi} = \underset{\phi}{\operatorname{argmin}} [L[\phi]] = \underset{\phi}{\operatorname{argmin}} \left[- \sum_{i=1}^I \log [Pr(\mathbf{y}_i|\mathbf{f}[\mathbf{x}_i, \phi])] \right]. \quad (5.7)$$

$$L[\phi] = - \sum_{i=1}^I \log [\text{softmax}_{y_i} [\mathbf{f}[\mathbf{x}_i, \phi]]]$$

$$\text{softmax}_k[\mathbf{z}] = \frac{\exp[z_k]}{\sum_{k'=1}^K \exp[z_{k'}]}$$

$$= - \sum_{i=1}^I f_{y_i} [\mathbf{x}_i, \phi] - \log \left[\sum_{k=1}^K \exp [f_k [\mathbf{x}_i, \phi]] \right]$$

Multiclass cross-entropy loss

49